

5.1 Hilbert Spaces

Inner-product spaces as the setting for kernels.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

The figures follow Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning*, Theodoridis, and Géron, *Hands-On Machine Learning*.

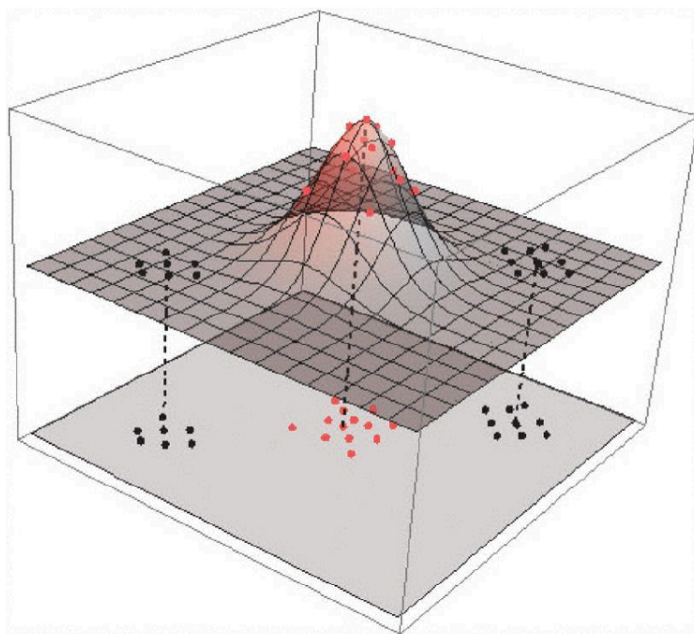
1. Central idea

Many datasets live on a **nonlinear** set. One response is to map the inputs to a new space where the same task becomes **linear**. Choosing that map is not obvious.

Given pairs (y_n, x_n) for $n = 1, \dots, N$, apply $\phi: \mathbb{R}^l \rightarrow \mathbb{R}^K$ with $K \gg l$ so that the images $\phi(x_n)$ are linearly separable in $\mathbb{R}^K$.

Points that cannot be separated by a line in the original plane can be separated by a plane after a nonlinear map into 3-space. You have embedded an l -dimensional manifold in a K -dimensional space so the two classes become linearly separable.

Working in that high-dimensional space is expensive: many parameters, computational cost, and a real risk of **overfitting**.



2. Kernels and RKHS

Instead of writing ϕ and computing inner products of $\phi(x_i)$ and $\phi(x_j)$, define a similarity $\kappa(x_i, x_j)$. For a class of similarities called **kernels**, that function **implicitly** defines a feature map ϕ .

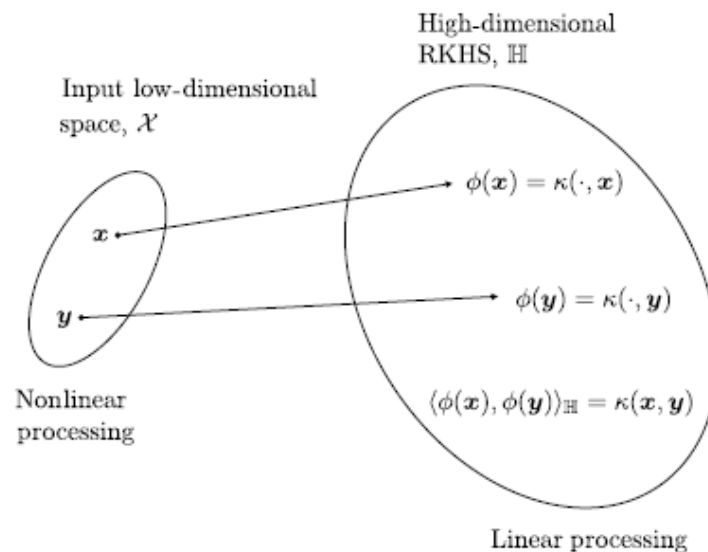
Let $\mathbb{H}$ be a Hilbert space of real-valued functions: an inner product, and completeness in the induced norm. $\mathbb{H}$ is a **reproducing kernel Hilbert space (RKHS)** if there is a kernel $\kappa: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ such that every $f \in \mathbb{H}$ and every $x \in \mathcal{X}$ satisfy $f(x) = \langle f, \kappa(\cdot, x) \rangle$. In this course, think $\mathcal{X} = \mathbb{R}^l$ and $\mathbb{H} = \mathbb{R}^K$.

Each kernel κ has a unique RKHS (Aronszajn, 1950). The **canonical feature map** is $\phi(x) = \kappa(\cdot, x)$, and $\mathbb{H}$ is the feature space.

The **kernel trick**: for any two points,

$$\langle \phi(x_i), \phi(x_j) \rangle = \kappa(x_i, x_j).$$

Inner products in $\mathbb{H}$ become a function evaluation in the original low-dimensional space.



The nonlinear task in the original space is linear in the RKHS. In practice: (a) write the algorithm using only inner products in the original space, then (b) replace those inner products by kernel evaluations.

The matrix $\mathbf{K} \in \mathbb{R}^{N \times N}$ of those pairwise evaluations is the **kernel matrix**. Every kernel matrix is **symmetric** and **positive semidefinite**.

Choose the kernel (and its parameters) by **nested cross-validation**. The specific formula for κ does not enter the derivation. After you have the predictor, you can swap kernels; each choice is a different nonlinearity.

3. Representer theorem

The **representer theorem** lets you optimize an empirical loss over a finite sample even when f lives in a very high-dimensional (even infinite-dimensional) RKHS.

Theorem. Let $\Omega: [0, \infty) \rightarrow \mathbb{R}$ be strictly increasing, and let $\mathcal{L}: \mathbb{R}^2 \rightarrow \mathbb{R}$ be any loss. Each minimizer $f \in \mathbb{H}$ of

$$\min_{f \in \mathbb{H}} J(f) = \sum_{n=1}^N \mathcal{L}(y_n, f(x_n)) + \lambda \Omega(\|f\|^2)$$

has the form

$$f(\cdot) = \sum_{n=1}^N \theta_n \kappa(\cdot, x_n),$$

with $\theta_n \in \mathbb{R}$.

To minimize J you plug in that expansion and optimize only the **finite** list $\theta_1, \dots, \theta_N$.

4. Practice

1. What does the kernel trick replace an inner product in $\mathbb{H}$ with?
2. Why is working with an explicit high-dimensional map ϕ expensive?
3. What does the representer theorem say you may optimize instead of the function $f \in \mathbb{H}$?